SIMATS ENGINEERING

ASSESSMENT TOOL 2: Scenario-Based

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO2: *Analyze virtualization architectures to identify performance and security issues in cloud computing environments.* (BL3)

Assessment Tool Weightage: 20%

Dave's Taxonomy: Manipulation (Level 2) **Total Marks: 25**

Question Count: 5

Code of Conduct

I __G.ALTHAF (192372004) _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _G.ALTHAF_____

Criteria	Understanding of Scenario (1)	Identification of Risks / Issues (1)	Application of Concepts (1.25)	Decision Justification (1)	Clarity (075)	TOTAL (5)
Weightage	**20%**	**20%**	**25%**	**20%**	**15%**	**100%**
Q1						
Q2						
Q3						
Q4						
Q5						

Signature of the Faculty Total Marks Awarded:

Scenario-Based Questions

1. A financial institution plans to deploy a private cloud with strict compliance requirements. Recommend an appropriate cloud datacenter architecture and explain how virtualization and isolation mechanisms can ensure security and regulatory compliance.
2. A SaaS provider experiences inconsistent application performance due to noisy neighbor effects in a virtualized environment. Analyze the root cause and propose resource allocation and hypervisor-level solutions.
3. An organization wants to migrate legacy monolithic applications into a Service-Oriented Architecture (SOA) in the cloud. Explain the migration strategy and how SOA improves scalability and maintainability.
4. A government agency is implementing identity federation across multiple departments using cloud services. Explain how identity management, authentication, and privacy policies should be designed to ensure secure access.
5. A cloud provider wants to implement real-time user presence detection for collaborative applications. Discuss how presence services can be integrated into cloud architecture while maintaining user privacy and system scalability.

Solutions:-

Question 1: Financial Institution Private Cloud Architecture

1. Understanding of Scenario

A financial institution requires a private cloud deployment to handle highly sensitive financial data, transactions, and user operations. Because it operates within a heavily regulated sector (governed by standards like PCI-DSS, SOC 2, and GDPR), the architecture must guarantee data residency, absolute data isolation, auditability, and defense-in-depth security.

2. Identification of Risks / Issues

- Shared-Fate Multi-Tenancy Risks: If logical isolation fails at the hypervisor layer, cross-VM data leaks can occur.
- Compliance Violations: Non-compliance with strict financial standards due to unauthorized access, unencrypted data transit, or lack of granular audit trails.
- Hypervisor Vulnerabilities: "VM Escape" attacks, where an attacker breaks out of a guest VM to access the host hypervisor or other co-located VMs.

3. Application of Concepts

- Architecture Recommendation: A Multi-Zone Private Cloud Architecture built on a Next-Generation Software-Defined Datacenter (SDDC) framework (e.g., VMware NSX/vSphere or OpenStack with KVM), integrated with a Hardware Security Module (HSM).
- Virtualization & Isolation Mechanisms:
    - Compute Isolation: Deploying Single-Tenant Bare-Metal Hypervisors combined with hardware-assisted virtualization (Intel VT-x / AMD-V) and CPU pinning.
    - Network Isolation: Implementing Micro-segmentation using distributed firewalls at the virtual NIC (vNIC) level, creating zero-trust boundaries between workloads.
    - Storage Isolation: Implementing dedicated storage LUNs with cryptographic isolation using AES-256 bit encryption for data-at-rest managed via external HSM keys.

4. Decision Justification

- Why Micro-segmentation? It prevents lateral movement of threats within the datacenter; if a web server VM is compromised, the database VM remains secure behind its own isolated segment.
- Why Single-Tenant Hypervisors? It completely eliminates the regulatory risk of co-mingling financial applications with non-compliant workloads, meeting strict PCI-DSS requirements.
- Why Cryptographic Isolation? It guarantees that even if physical storage drives are decommissioned or stolen, data remains completely unreadable, fulfilling strict privacy laws.

Question 2: SaaS Provider Noisy Neighbor Mitigation

1. Understanding of Scenario

A Software-as-a-Service (SaaS) provider is experiencing unpredictable application performance fluctuations. In this shared, virtualized infrastructure, multiple tenant VMs compete for finite underlying physical hardware resources (CPU, Memory, Disk I/O, Network bandwidth).

2. Identification of Risks / Issues

- Noisy Neighbor Effect: A single high-demand tenant monopolizes shared physical resources, starving adjacent tenant VMs.
- SLA Breaches: Degradation in application response times leads to breaches of Service Level Agreements (SLAs) and loss of customer trust.
- Resource Contention: Uncontrolled hypervisor-level scheduling causing CPU scheduling delays (high CPU ready time) and storage I/O bottlenecks.

3. Application of Concepts

- Root Cause Analysis: The hypervisor is overcommitting resources without strict boundaries. When Tenant A runs heavy batch jobs, the hypervisor fails to dynamically constrain its physical resource consumption, directly impacting Tenant B.
- Resource Allocation Solutions:
    - Hard Limits and Reservations: Setting explicit CPU and Memory Reservations (guaranteed minimum allocations) and Limits (hard caps) for each tenant VM.
    - Storage I/O Control (SIOC): Implementing Quality of Service (QoS) throttles on IOPS (Input/Output Operations Per Second) at the datastore level.
- Hypervisor-Level Solutions:
    - Dynamic Resource Scheduling (DRS): Configuring live migration tools (like VMware vMotion) to automatically move noisy tenants to underutilized physical hosts.
    - Affinity / Anti-Affinity Rules: Enforcing hypervisor rules that ensure high-performance tenant VMs are never co-located on the same physical host.

4. Decision Justification

- Why Reservations and Limits? They provide predictable performance baselines, ensuring no single tenant can starve others, directly mitigating the noisy neighbor effect.
- Why Storage QoS? Storage is frequently the primary bottleneck in SaaS databases; capping IOPS ensures fair-share distribution of disk performance.
- Why Live Migration (DRS)? It continuously balances the cluster workload proactively, removing the administrative burden of manual resource balancing.

Question 3: Legacy Monolithic to SOA Migration

1. Understanding of Scenario

An organization seeks to modernize its rigid, single-tier, legacy monolithic application by refactoring it into a cloud-native Service-Oriented Architecture (SOA) or Microservices framework. This addresses the bottleneck of scaling the entire monolith when only a single component requires more resources.

2. Identification of Risks / Issues

- Operational Disruption: Risks of prolonged system downtime during database and logic migration.
- Distributed Complexity: Transitioning to a distributed environment introduces network latency, data consistency issues, and complex debugging.
- Tight Coupling: Finding hidden dependencies within the monolith that make breaking it apart challenging.

3. Application of Concepts

- Migration Strategy: The Strangler Fig Application Strategy. Instead of a risky "big-bang" rewrite, legacy features are progressively replaced with new SOA services, routed through an API Gateway, until the old monolith is completely deprecated.
- How SOA Improves Scalability:
    - Horizontal, Granular Scaling: Instead of duplicating the entire monolith, only high-demand components (e.g., a payment service) are scaled horizontally across multiple cloud instances.
    - Independent Deployment: Teams can modify, scale, and update specific services without deploying or risking the stability of the entire system.
- How SOA Improves Maintainability:
    - Decoupled Architecture: Services communicate asynchronously via standard protocols (REST APIs, gRPC, or Message Queues like RabbitMQ).
    - Fault Isolation: If a single service fails (e.g., a recommendation engine), the core application (e.g., order processing) remains fully functional.

4. Decision Justification

- Why Strangler Fig Strategy? It minimizes business risk by allowing continuous feature delivery and testing during the cloud migration phase.
- Why API Gateway Routing? It abstracts the backend architecture from the users, ensuring seamless transitions while the backend moves from monolithic endpoints to SOA endpoints.
- Why Decoupled Services? It fosters high agility; individual engineering teams can own specific services and update them using modern CI/CD pipelines without cross-team bottlenecks.

Question 4: Government Agency Identity Federation

1. Understanding of Scenario

A government agency requires cross-departmental collaboration using diverse cloud services. They need a unified identity system where an employee from Department A can securely access authorized resources in Department B without creating separate credentials for each environment.

2. Identification of Risks / Issues

- Credential Proliferation: Users managing dozens of passwords, increasing the likelihood of weak passwords and credential harvesting.
- Privilege Escalation & Sprawl: Unauthorized users gaining access to classified departmental data due to poorly mapped roles.
- Privacy and Data Leaks: Accidental exposure of personally identifiable information (PII) across departmental boundaries.

3. Application of Concepts

- Identity Management Architecture: Implement a Centralized Identity Provider (IdP) using open federation standards such as SAML 2.0 or OpenID Connect (OIDC) / OAuth 2.0.
- Authentication Framework:
    - Single Sign-On (SSO): Users authenticate once at the home IdP, obtaining cryptographic tokens accepted by relying party cloud services.
    - Phishing-Resistant MFA: Enforcing hardware tokens (FIDO2/WebAuthn) or biometric verification for all cross-departmental access.
- Privacy & Access Control Policies:
    - Attribute-Based Access Control (ABAC): Access decisions are evaluated dynamically based on user attributes (e.g., clearance level), environmental attributes (e.g., network location), and resource types.
    - Data Minimization via Token Scopes: Sharing only the bare minimum user attributes required for access verification, preventing cross-departmental PII tracking.

4. Decision Justification

- Why SAML/OIDC Federation: - It eliminates the need to synchronize actual user passwords between departments. The identity provider merely passes signed XML/JSON tokens, keeping credentials secure.
- Why ABAC over RBAC Government compliance requires highly granular controls; ABAC allows conditions like "Allow access *only if* the user is from Department A, holds Top-Secret clearance, and is accessing from a secure government network."
- Why Data Minimization It meets legal privacy mandates by shielding citizen and employee data from unauthorized departmental visibility.

Question 5: Real-Time User Presence Detection in Cloud Architecture

1. Understanding of Scenario

A cloud provider wants to integrate real-time presence detection (e.g., "Online", "Away", "Busy") into collaborative applications (like document editing or chat tools). The system must handle millions of concurrent active users updates instantly while fiercely protecting user privacy.

2. Identification of Risks / Issues

- Scalability Bottlenecks: High-frequency "heartbeat" updates from millions of clients can easily overwhelm traditional relational databases and application servers.
- Privacy Violations: Continuous user status tracking can expose behavioral patterns, exact working hours, or location data without explicit user consent.
- Network Overhead: Persistent open connections consuming high memory and bandwidth across the system.

3. Application of Concepts

- Cloud Architecture Integration:
    - Persistent Connections: Utilize WebSockets or gRPC backed by an API Gateway managing persistent bi-directional communication channels.
    - In-Memory State Store: Utilizing an in-memory database like Redis Cluster configured with pub/sub (Publish/Subscribe) capabilities to cache and broadcast state changes instantly.
    - Event-Driven Processing: Routing massive status changes through message brokers like Apache Kafka to handle peaks gracefully.
- Privacy Implementation:
    - Granular Presence Controls: Giving users absolute control to mask their status ("Go Incognito", "Appear Offline").
    - Data Ephemerality: Presence data should strictly live in memory with low Time-To-Live (TTL) values, ensuring no historical log of a user's minute-by-minute activity is archived.
- Scalability Enhancements: Decoupling the write path (updating status in Redis) from the read path (distributing status to active friends via Pub/Sub).

4. Decision Justification

- Why Redis Pub/Sub? Disk-based databases would crash under millions of state updates per second. Redis processes operations in-memory at sub-millisecond speeds, scaling effortlessly.
- Why WebSockets over HTTP Polling? HTTP polling creates massive unnecessary network overhead. WebSockets hold a single lightweight, open channel that pushes data only when a status change actually occurs.
- Why Ephemeral Data (TTL)? It mitigates privacy risks entirely. If a presence status expires automatically in 60 seconds without being written to persistent storage, it cannot be leaked or subpoenaed later.